INFORME DE LABORATORIO

INFORMACIÓN BÁSICA

ASIGNATURA:	INTRODUCCION AL DESARROLLO DE NUEVAS PLATAFORMAS (E)				
TÍTULO DE LA PRÁCTICA:	Tarjetas de lugares con Row, Column, Box y Modifier en Jetpack Compose				
NÚMERO DE LA PRÁCTICA:	06	**AÑO LECTIVO:**	2026	**NRO. SEMESTRE:**	B
FECHA DE PRESENTACIÓN:	06/10/2026	**HORA DE PRESENTACIÓN:**	11:59:00		
INTEGRANTE(s): - Hancco Mullisaca, Sergio Danilo - Huacani Jara, Denise Andrea				**NOTA (0 - 20):**	Nota colocada por el docente
DOCENTE: Roxana Evelyn Limache Calatayud					

SOLUCIÓN Y RESULTADOS

I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS

Link Repositorio (Ambas propuestas): https://github.com/shanccom/IDNP_Repo/tree/main/lab_06_TarjetasCampus_Compose

Ejercicio Propuesto: Tarjetas de Campus con Jetpack Compose

Interfaz en Android que presenta lugares del campus con dos patrones de tarjetas en **Jetpack Compose**: horizontal con `Row` (`TarjetaLugar`) y apilada con imagen de fondo y scrim usando `Box` (`TarjetaCompacta`).

Paso 1. Creación del Proyecto en Android Studio

Proyecto **Empty Activity** (Compose + Material 3) llamado `TarjetasCampus_Compose` en Kotlin.

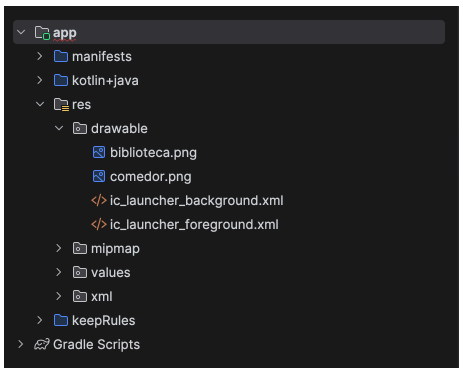
Paso 2. Importaciones necesarias

Importaciones necesarias para maquetación y diseño:

```
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.Image
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.painter.Painter
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.example.tarjetascampus_compose.ui.theme.TarjetasCampus_ComposeTheme
```

Paso 3. Recursos Gráficos en `res/drawable`

Recursos visuales del campus (`biblioteca.png` y `comedor.png`) en `res/drawable`:



Paso 4. Declaración de `TarjetaLugar`

Función `@Composable TarjetaLugar` con sus parámetros (`nombre`, `descripcion`, `imagen`, `colorFondo`):

```
@Composable
fun TarjetaLugar(
    nombre: String,
    descripcion: String,
    imagen: Painter,
    colorFondo: Color
) {
// el contenido se agrega en el Paso 5
}
```

Paso 5. Envoltorio `Surface` y estructura `Row`

Contenedor con esquinas de `24.dp` y fila centrada:

```
@Composable
fun TarjetaLugar(
    nombre: String,
    descripcion: String,
    imagen: Painter,
    colorFondo: Color
) {
    Surface(
        shape = RoundedCornerShape( size = 24.dp),
        color = colorFondo
    ) {
        Row(
            verticalAlignment = Alignment.CenterVertically,
            modifier = Modifier.padding( all = 16.dp)
        ) {
// el contenido se agrega en el Paso 6
        }
    }
}
```

Paso 6. Contenido Completo de `TarjetaLugar`

Imagen circular con `Modifier.clip(CircleShape)` y textos con `FontWeight.Bold`:

```
Row(
    verticalAlignment = Alignment.CenterVertically,
    modifier = Modifier.padding( all = 16.dp)
) {
    Image(
        painter = imagen,
        contentDescription = nombre,
        contentScale = ContentScale.Crop,
        modifier = Modifier
            .size(90.dp)
            .clip(CircleShape)
    )
    Spacer(modifier = Modifier.width(16.dp))
    Column {
        Text(
            text = nombre,
            fontWeight = FontWeight.Bold,
            fontSize = 20.sp
        )
        Text(
            text = descripcion,
            fontSize = 14.sp
        )
    }
}
```

Paso 7. Llamado en lista (`ListaTarjetasLugar`)
Lista vertical con `Column` y separación de `12.dp`:

```
@Composable
fun ListaTarjetasLugar() {
    Column(verticalArrangement = Arrangement.spacedBy(12.dp)) {
        TarjetaLugar(
            nombre = "Biblioteca Central",
            descripcion = "Zona de estudio y préstamo de libros",
            imagen = painterResource(id = R.drawable.biblioteca),
            colorFondo = Color(0xFFE3F2FD)
        )
        TarjetaLugar(
            nombre = "Comedor Universitario",
            descripcion = "Servicio de alimentación para estudiantes",
            imagen = painterResource(id = R.drawable.comedor),
            colorFondo = Color(0xFFFFF3E0)
        )
    }
}
```

Paso 8. Estructura base de `TarjetaCompacta`
Tarjeta apilada con `Box`, altura `140.dp` e imagen de fondo:

```
@Composable
fun TarjetaCompacta(
    nombre: String,
    imagen: Painter
) {
    Box(
        modifier = Modifier
            .fillMaxWidth()
            .height(140.dp)
            .clip( shape = RoundedCornerShape( size = 16.dp))
    ) {
        Image(
            painter = imagen,
            contentDescription = nombre,
            contentScale = ContentScale.Crop,
            modifier = Modifier.matchParentSize()
        )
// el scrim y el texto se agregan en el Paso 9
    }
}
```

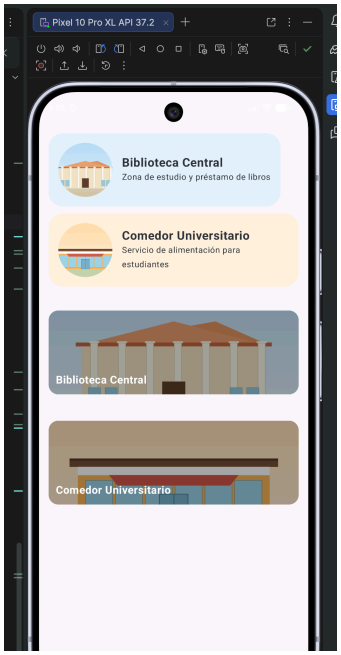
Paso 9. Superposición y Scrim en `TarjetaCompacta`

Capa oscura semitransparente (alfa `0.35f`) y texto blanco en `BottomStart`:

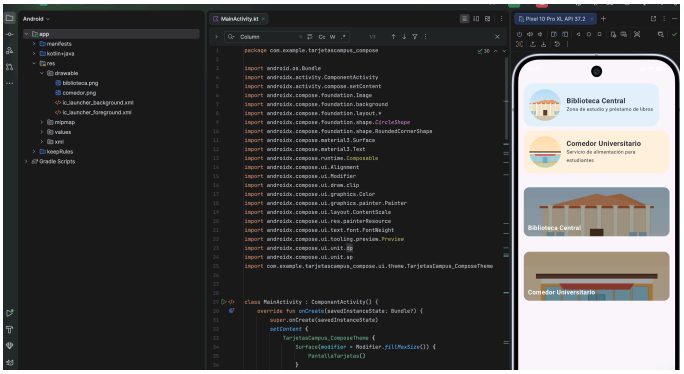
```
@Composable
fun TarjetaCompacta(
    nombre: String,
    imagen: Painter
) {
    Box(
        modifier = Modifier
            .fillMaxWidth()
            .height(140.dp)
            .clip( shape = RoundedCornerShape( size = 16.dp))
    ) {
        Image(
            painter = imagen,
            contentDescription = nombre,
            contentScale = ContentScale.Crop,
            modifier = Modifier.matchParentSize()
        )
        Box(
            modifier = Modifier
                .matchParentSize()
                .background(Color.Black.copy(alpha = 0.35f))
        )
        Text(
            text = nombre,
            color = Color.White,
            fontWeight = FontWeight.Bold,
            fontSize = 18.sp,
            modifier = Modifier
                .align(Alignment.BottomStart)
                .padding( all = 12.dp)
        )
    }
}
```

Paso 10 y 11. Demostración y Ejecución de la Aplicación

Ambos estilos integrados en `PantallaTarjetas` y ejecutados en el emulador:



Evidencia 1: Vista en el emulador



Evidencia 2: Android Studio y ejecución en simultáneo

Criterio	Estilo 1: `TarjetaLugar` (Row)	Estilo 2: `TarjetaCompacta` (Box)
Layout Base	`Surface` + `Row` + `Column`	`Box` (superposición en eje Z)
Tratamiento de Imagen	Circular (`CircleShape`), fija `90.dp`	Fondo completo (`matchParentSize`), recorte `16.dp`
Legibilidad	Texto sobre fondo pastel claro	Texto blanco sobre imagen con scrim oscuro (35%)
Resultado Visual	Ficha informativa clara con descripción	Banner compacto de alto impacto visual

Ejercicio 1: Composable DetalleLugarScreen

Crea un Composable DetalleLugarScreen(nombre: String, descripcion: String, imagen: Painter) que muestre la información del lugar a tamaño completo, reutilizando el estilo de TarjetaLugar.

Primera y Segunda Propuesta

Explicación y Reutilización de Estilos

Se reutiliza la línea gráfica de `TarjetaLugar`: `Surface` con esquinas de `24.dp`, imagen a tamaño completo (`fillMaxWidth()`, `240.dp`, `ContentScale.Crop`), nombre en negrita `24.sp` y descripción `16.sp`. Arriba, una `Row` con botón `ArrowBack` que invoca `onVolver`.

Implementación del Composable

Código de la pantalla de detalle:

```
@Composable
fun DetalleLugarScreen(
    nombre: String,
    descripcion: String,
```

```
    imagen: Painter,
    onVolver: () -> Unit = {}
) {
    Column(
        modifier = Modifier
            .fillMaxSize()
            .statusBarsPadding()
            .padding(16.dp)
    ) {
        Row(verticalAlignment = Alignment.CenterVertically) {
            IconButton(onClick = onVolver) {
                Icon(
                    imageVector = Icons.AutoMirrored.Filled.ArrowBack,
                    contentDescription = "Volver"
                )
            }
            Text(
                text = "Detalle del lugar",
                style = MaterialTheme.typography.titleMedium
            )
        }

        Spacer(modifier = Modifier.height(16.dp))

        Surface(
            shape = RoundedCornerShape(24.dp),
            color = MaterialTheme.colorScheme.surfaceVariant,
            modifier = Modifier.fillMaxWidth()
        ) {
            Column {
                Image(
                    painter = imagen,
                    contentDescription = nombre,
                    contentScale = ContentScale.Crop,
                    modifier = Modifier
                        .fillMaxWidth()
                        .height(240.dp)
                )

                Column(modifier = Modifier.padding(16.dp)) {
                    Text(
                        text = nombre,
                        fontWeight = FontWeight.Bold,
                        fontSize = 24.sp
                    )
                    Spacer(modifier = Modifier.height(8.dp))
                    Text(
                        text = descripcion,
                        fontSize = 16.sp
                    )
                }
            }
        }
```

```
    }
}
```

Ejercicio 2: Envuelve TarjetaLugar y TarjetaCompacta con Modifier.clickable { ... } y, usando el mismo patrón de navegación del Laboratorio 05 (navController.navigate(...) o una lambda), lleva al usuario a DetalleLugarScreen al pulsar una tarjeta.

Primera y Segunda Propuesta

Paso 1. Integración de `Modifier.clickable` en `TarjetaLugar`

`TarjetaLugar` recibe `onClick: () -> Unit = {}` y lo conecta en el `Surface` raíz con `.clickable(onClick = onClick)`:

```
@Composable
fun TarjetaLugar(
    nombre: String,
    descripcion: String,
    imagen: Painter,
    colorFondo: Color,
    onClick: () -> Unit = {}
) {
    Surface(
        shape = RoundedCornerShape(24.dp),
        color = colorFondo,
        modifier = Modifier
            .fillMaxWidth()
            .clickable(onClick = onClick)
    ) {
        // Contenido con imagen circular, textos y espaciadores
    }
}
```

Paso 2. Integración de `Modifier.clickable` en `TarjetaCompacta`

Mismo caso en `TarjetaCompacta`: `.clickable(onClick = onClick)` justo después de `.clip(RoundedCornerShape(16.dp))` en el `Box` raíz:

```
@Composable
fun TarjetaCompacta(
    nombre: String,
    descripcion: String = "",
    imagenRes: Int,
    onClick: () -> Unit = {}
) {
    Box(
        modifier = Modifier
            .fillMaxWidth()
            .height(140.dp)
            .clip(RoundedCornerShape(16.dp))
            .clickable(onClick = onClick)
    ) {
        // Imagen de fondo, scrim oscuro y textos superpuestos
    }
}
```

Paso 3. Patrón de Navegación con Lambdas y `navController.navigate`

Las tarjetas despachan la lambda `onAbrirDetalle: (String, String, Int) -> Unit` hacia el nivel superior:

```
// Invocación en PantallaTarjetas (Home)
TarjetaCompacta(
    nombre = "Biblioteca Central",
    descripcion = "Zona de estudio y préstamo de libros",
    imagenRes = R.drawable.biblioteca,
    onClick = { onAbrirDetalle("Biblioteca Central", "Zona de estudio y préstamo de libros",
R.drawable.biblioteca) }
)

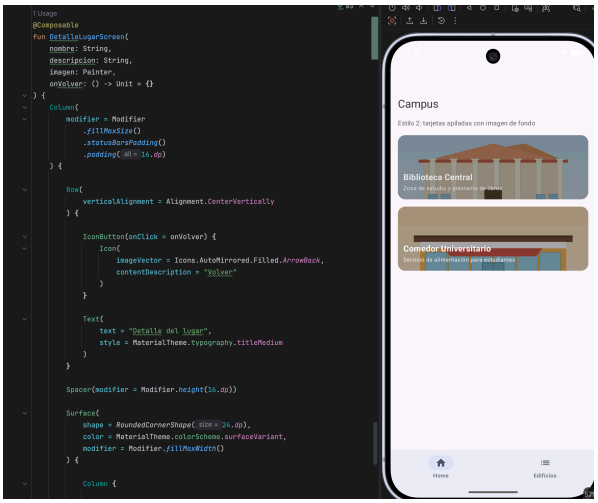
// Invocación en EdificiosScreen
TarjetaLugar(
    nombre = lugar.nombre,
    descripcion = lugar.descripcion,
    imagen = painterResource(id = lugar.imagenRes),
    colorFondo = lugar.colorFondo,
    onClick = { onAbrirDetalle(lugar.nombre, lugar.descripcion, lugar.imagenRes) }
)
```

En `CampusApp`, el `NavHost` ejecuta la transición a `DetalleLugarScreen`:

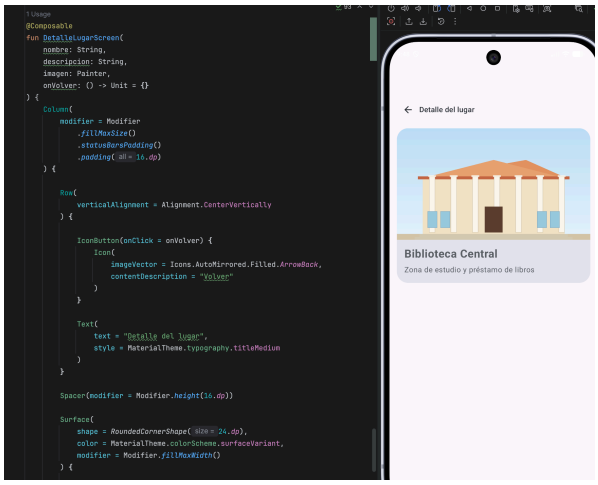
```
PantallaTarjetas(
    onAbrirDetalle = { nombre, descripcion, imagenRes ->
        navController.navigate(Screen.Detalle.crearRuta(nombre, descripcion, imagenRes))
    }
)
```

Evidencias de Navegación

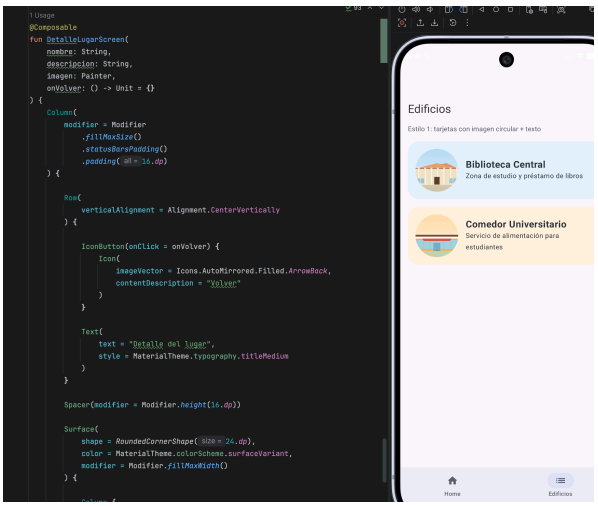
Flujo en el emulador: al pulsar una tarjeta se navega a `DetalleLugarScreen`:



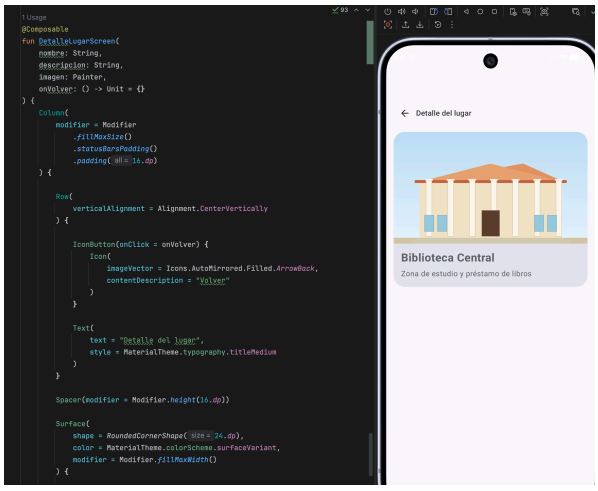
Evidencia 1: Lista interactiva de Edificios



Evidencia 2: Navegación al detalle del Pabellón A



Evidencia 3: Navegación al detalle del Auditorio



Evidencia 4: Navegación al detalle del Comedor

Ejercicio 3: Reemplaza la lista simple de EdificiosScreen (nombre + botón) por una lista de TarjetaLugar o TarjetaCompacta con datos de al menos 4 edificios del campus.

Primera propuesta

La `Row` de nombre + botón de `EdificiosScreen` se reemplazó por tarjetas reutilizables, compartiendo el lenguaje visual del Home.

Paso 1. Modelo de Datos de los Edificios

Estructura `Lugar` con los atributos que necesitan las tarjetas:

```
data class Lugar(
    val nombre: String,
    val descripcion: String,
    val imagenRes: Int,
    val colorFondo: Color
)

val edificiosCampus = listOf(
    Lugar("Pabellón A", "Aulas y salas de clase del turno mañana", R.drawable.pabellon_a,
Color(0xFFBBDEFB)),
    Lugar("Biblioteca Central", "Zona de estudio y préstamo de libros", R.drawable.biblioteca,
Color(0xFFC8E6C9)),
    Lugar("Auditorio", "Eventos académicos y ceremonias institucionales", R.drawable.auditorio,
Color(0xFFFFE0B2)),
    Lugar("Comedor Universitario", "Servicio de alimentación para estudiantes",
R.drawable.comedor, Color(0xFFF8BBD0))
)
```

Paso 2. Reemplazo del Ítem Simple por `TarjetaLugar`

Cada ítem de la lista pasó de `Text` + `Button` a `TarjetaLugar`, reutilizando la lambda `onAbrirDetalle`:

```
@Composable
fun EdificiosScreen(
    onAbrirDetalle: (String, String, Int) -> Unit
```

```
) {
    LazyColumn(
        contentPadding = PaddingValues(16.dp),
        verticalArrangement = Arrangement.spacedBy(12.dp)
    ) {
        items(edificiosCampus) { lugar ->
            TarjetaLugar(
                nombre = lugar.nombre,
                descripcion = lugar.descripcion,
                imagen = painterResource(id = lugar.imagenRes),
                colorFondo = lugar.colorFondo,
                onClick = {
                    onAbrirDetalle(lugar.nombre, lugar.descripcion, lugar.imagenRes)
                }
            )
        }
    }
}
```

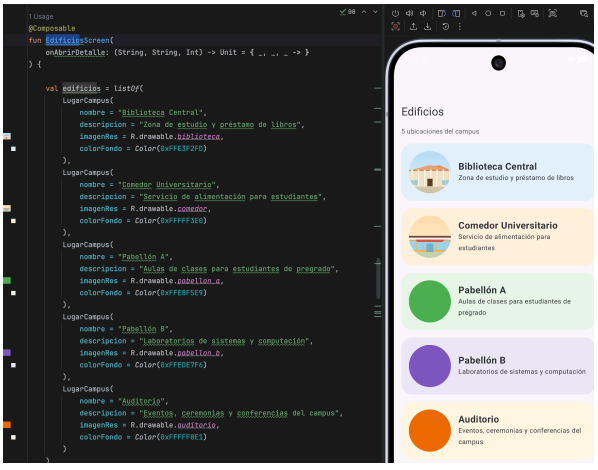
Paso 3. Versión Alternativa con `TarjetaCompacta`

La misma lista funciona con `TarjetaCompacta`, con idéntico clic y navegación:

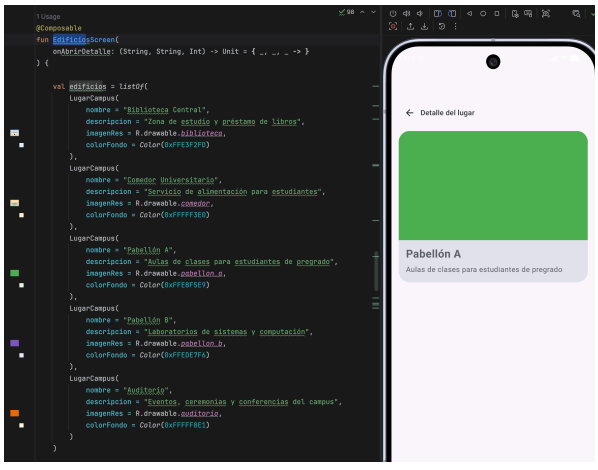
```
items(edificiosCampus) { lugar ->
    TarjetaCompacta(
        nombre = lugar.nombre,
        descripcion = lugar.descripcion,
        imagenRes = lugar.imagenRes,
        onClick = {
            onAbrirDetalle(lugar.nombre, lugar.descripcion, lugar.imagenRes)
        }
    )
}
```

Evidencias de la Lista de Edificios

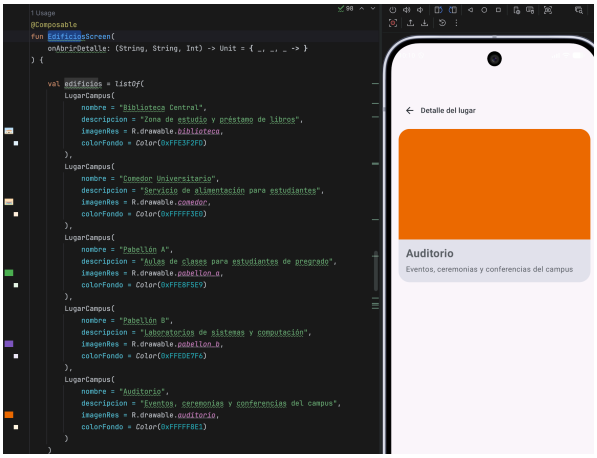
Resultado en el emulador con cuatro edificios como tarjetas y navegación al detalle:



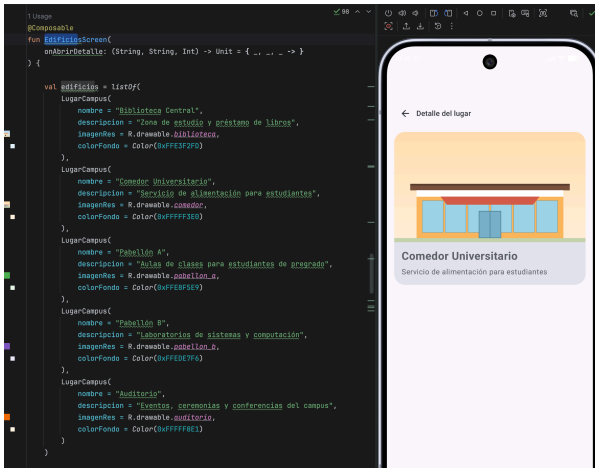
Evidencia 1: EdificiosScreen con lista de TarjetaLugar



Evidencia 2: Edificios del campus con TarjetaCompacta



Evidencia 3: Navegación al detalle desde la lista



Evidencia 4: Detalle de un edificio seleccionado

Segunda propuesta

En la segunda propuesta se implementó una interfaz enriquecida para `EdificiosScreen`, incorporando tarjetas con miniaturas ilustradas, etiquetas de distancia en kilómetros e indicadores de navegación, además de una barra de navegación inferior para la navegación global. Cada tarjeta interactiva permite acceder a la pantalla `DetalleLugarScreen` correspondiente.



Evidencia 1: Segunda propuesta — lista de edificios



Evidencia 2: Segunda propuesta — detalle de Biblioteca Central

(Reto opcional) Diseña un tercer estilo de tarjeta propio (por ejemplo con una etiqueta o ícono adicional) y documenta en el informe qué combinación de Row/Column/Box y Modifier utilizaste.

Primera propuesta

Se implementó `TarjetaEstilo3`, tarjeta minimalista con **etiqueta de distancia** e **ícono de flecha**, con la misma firma `onClick` de los estilos anteriores.

Paso 1. Estructura `Surface` como Contenedor Raíz

`Surface` raíz que concentra forma, color, elevación y borde teñido con `colorIcono` (alfa `0.18f`):

```
@Composable
fun TarjetaEstilo3(
    nombre: String,
    descripcion: String,
    etiqueta: String,
    imagenRes: Int,
    colorFondo: Color = MaterialTheme.colorScheme.surface,
    colorIcono: Color = MaterialTheme.colorScheme.primary,
    onClick: () -> Unit = {}
) {
    Surface(
        shape = RoundedCornerShape(18.dp),
        color = colorFondo,
        tonalElevation = 1.dp,
        shadowElevation = 2.dp,
        border = BorderStroke(1.dp, colorIcono.copy(alpha = 0.18f)),
        modifier = Modifier
            .fillMaxWidth()
            .clickable(onClick = onClick)
    ) {
        // Fila interna con imagen, textos, etiqueta e ícono
    }
}
```

Paso 2. Combinación `Row` + `Column` + `weight(1f)`

`Row` centrada con imagen, textos, etiqueta y flecha; `Modifier.weight(1f)` en el `Column` central reparte el espacio sobrante:

```
Row(
    modifier = Modifier.padding(horizontal = 14.dp, vertical = 14.dp),
    verticalAlignment = Alignment.CenterVertically
) {
    Image(
        painter = painterResource(id = imagenRes),
        contentDescription = nombre,
        contentScale = ContentScale.Crop,
        modifier = Modifier
            .size(52.dp)
            .clip(RoundedCornerShape(14.dp))   // miniatura cuadrada redondeada
    )

    Spacer(modifier = Modifier.width(12.dp))

    Column(
        modifier = Modifier.weight(1f),        // ocupa el espacio sobrante
```

```
        verticalArrangement = Arrangement.spacedBy(2.dp)
    ) {
        Text(text = nombre, fontWeight = FontWeight.SemiBold,
             fontSize = 15.sp, maxLines = 1)
        Text(text = descripcion, fontSize = 12.sp,
             color = MaterialTheme.colorScheme.onSurfaceVariant, maxLines = 1)
    }
}
```

Paso 3. Etiqueta «Píldora» con `Surface` Anidada

Etiqueta de distancia: `Surface` secundaria de `10.dp` con fondo translúcido, más el ícono flecha:

```
Surface(
    shape = RoundedCornerShape(10.dp),
    color = colorIcono.copy(alpha = 0.10f)
) {
    Text(
        text = etiqueta,                        // p. ej. "0.4 km"
        fontWeight = FontWeight.Medium,
        fontSize = 11.sp,
        color = colorIcono,
        modifier = Modifier.padding(horizontal = 10.dp, vertical = 5.dp)
    )
}

Spacer(modifier = Modifier.width(4.dp))

Icon(
    imageVector = Icons.AutoMirrored.Filled.ArrowForward,
    contentDescription = null,
    tint = MaterialTheme.colorScheme.onSurfaceVariant.copy(alpha = 0.45f),
    modifier = Modifier.size(16.dp)
)
```

Paso 4. Modifiers Utilizados (Resumen)

- `fillMaxWidth()` — ocupa el ancho disponible.
- `clickable(onClick = onClick)` — clic en toda el área.
- `clip(RoundedCornerShape(14.dp))` + `size(52.dp)` — miniatura redondeada.
- `padding(...)` — aire interno de la fila.
- `weight(1f)` — espacio sobrante para los textos.
- `Arrangement.spacedBy(...)` — separación vertical.

Paso 5. Invocación en `PantallaTarjetas`

Sección «Estilo 3 · minimalista» en el Home:

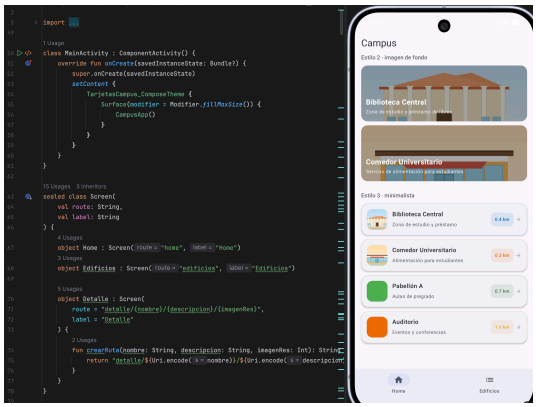
```
TarjetaEstilo3(
    nombre = "Biblioteca Central",
    descripcion = "Zona de estudio y préstamo",
    etiqueta = "0.4 km",
    imagenRes = R.drawable.biblioteca,
    colorIcono = Color(0xFF1565C0),
    onClick = {
        onAbrirDetalle(
```

```
            "Biblioteca Central",
            "Zona de estudio y préstamo de libros",
            R.drawable.biblioteca
        )
    }
)

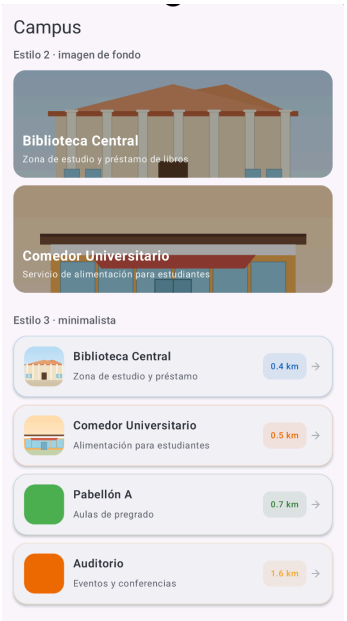
TarjetaEstilo3(
    nombre = "Comedor Universitario",
    descripcion = "Alimentación para estudiantes",
    etiqueta = "0.5 km",
    imagenRes = R.drawable.comedor,
    colorIcono = Color(0xFFEF6C00),
    onClick = { onAbrirDetalle("Comedor Universitario", "Servicio de alimentación para
estudiantes", R.drawable.comedor) }
)
```

Evidencias del Tercer Estilo

Resultado en el emulador de la sección «Estilo 3 · minimalista»:

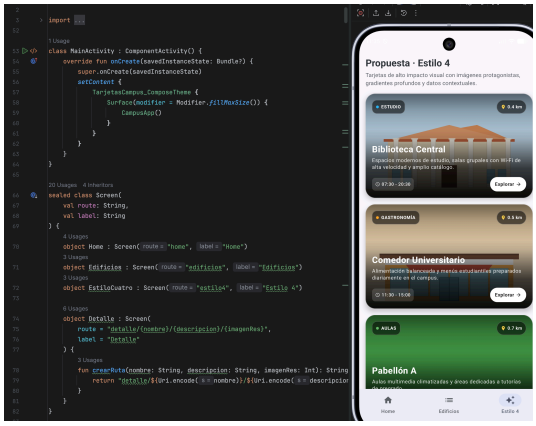


Evidencia 1: Sección «Estilo 3 · minimalista» en el Home



Evidencia 2: Tarjetas con etiqueta de distancia e ícono de flecha

Segunda propuesta



Evidencia 1: EdificiosScreen con lista de TarjetaLugar



Evidencia 1: EdificiosScreen con lista de TarjetaLugar

II. SOLUCIÓN DEL CUESTIONARIO

Pregunta 1: ¿Lograste resolver completamente el Ejercicio/Problema Resuelto por el Docente (dos estilos de tarjeta de lugares: una con Row/Column mostrando imagen y texto, y otra apilada con Box y superposición de texto)? Adjunta una captura de pantalla del emulador mostrando el resultado como evidencia.

Respuesta 1

Sí, logré resolverlo. En el primer estilo, usando Row y Column pude organizar la imagen junto al texto de manera ordenada y clara. En el segundo, con Box pude apilar la imagen y superponer el texto sobre ella, logrando el efecto visual que se pedía. Ambas tarjetas se muestran correctamente en el emulador, tal como se esperaba.

Respuesta 2

Sí, también lo resolví completamente. El estilo con Row/Column me permitió colocar la imagen a la izquierda y el texto a la derecha con buena alineación, y el estilo apilado con Box logró el efecto de texto sobre la imagen con la ayuda de alignment y padding. Ambas tarjetas compilan y se visualizan sin problemas en el emulador.

Evidencias (ambas respuestas)

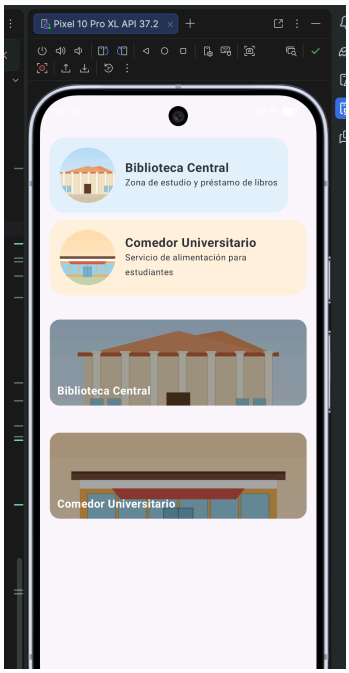


Figura 1: Evidencia 1 - Primer estilo de tarjeta

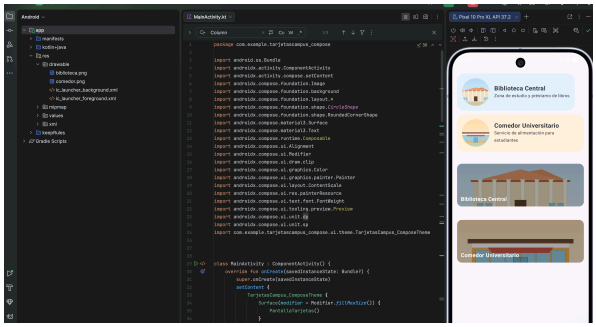


Figura 2: Evidencia 2 - Segundo estilo de tarjeta

Pregunta 2: ¿Qué combinación de Composables (Row, Column, Box) y Modifiers consideras que fue la mejor opción para cada uno de los dos estilos de tarjeta implementados? Justifica tu respuesta comparando ambos casos.

Respuesta 1

Para la primera tarjeta, considero que la mejor combinación fue Row para colocar la imagen a un lado y Column para organizar el título y la descripción verticalmente. Esto permitió que el contenido fuera legible y mantuviera una estructura horizontal equilibrada. Para la segunda tarjeta, Box fue la opción más adecuada, ya que nos permite apilar elementos y con align o padding logramos superponer el texto sobre la imagen sin romper el diseño. En comparación, la primera prioriza la organización jerárquica por filas y columnas, mientras que la segunda busca un enfoque más visual y centrado en la imagen.

Respuesta 2

Coincido en que Row + Column es la mejor opción para el primer estilo: Row resuelve la disposición horizontal de imagen y texto, y Column apila título y descripción; los Modifiers `padding`, `size` y `clip(CircleShape)` dan espaciado y forma a la miniatura. Para el segundo estilo, Box es superior porque aprovecha la superposición en el eje Z: la imagen ocupa todo el contenedor con `matchParentSize` y el texto se ubica con `align(Alignment.BottomStart)`, mientras un scrim con `background(Color.Black.copy(alpha = 0.35f))` garantiza legibilidad. La diferencia clave es que el primer estilo prioriza la lectura de datos y el segundo el impacto visual.

Pregunta 3: ¿Qué diferencia encontraste entre construir estas interfaces con Composables y Modifier, en comparación con el enfoque clásico de LinearLayout/ConstraintLayout en XML? Menciona al menos una ventaja concreta que hayas notado.

Respuesta 1

La principal diferencia es que en Compose todo es declarativo, es decir, se describe cómo debe verse la interfaz y el sistema se encarga de actualizarla, mientras que en XML con LinearLayout o ConstraintLayout es más imperativo y hay que lidiar con

referencias y jerarquías más rígidas. Además, con Composables y Modifier se puede aplicar estilo, espaciado y alineación de manera más directa y reutilizable, sin necesidad de crear múltiples archivos o atributos anidados. Una ventaja concreta que noté fue la rapidez para hacer cambios: al modificar un Composable, el preview o la compilación es más ágil y puedo ver el resultado casi al instante, lo cual facilita mucho el proceso de diseño.

Respuesta 2

La diferencia principal es de paradigma: en XML describimos una jerarquía de vistas estática que hay que actualizar manualmente, mientras que en Compose describimos el estado y la interfaz se redibuja sola. Los Modifiers reemplazan buena parte de los atributos XML (`layout_margin`, `padding`, `background`, `cornerRadius`) encadenados en una sola línea y reutilizables entre Composables. Una ventaja concreta que noté fue la reutilización: una misma función `TarjetaLugar` se invoca en distintas pantallas sin duplicar layouts, algo más engorroso de lograr con `include` y `merge` en XML.

III. CONCLUSIONES Y RECOMENDACIONES

Conclusiones

- Jetpack Compose simplifica la creación de interfaces gracias a su enfoque declarativo, lo que permite construir diseños como las tarjetas de lugares de manera más rápida e intuitiva.
- El uso adecuado de Row, Column y Box junto a Modifiers permite lograr distintos estilos visuales, desde diseños horizontales hasta superposiciones, adaptados a las necesidades del proyecto.
- En comparación con XML, Compose reduce la cantidad de código y agiliza el proceso de prueba con Previews, lo que mejora la experiencia de desarrollo.
- Esta herramienta resulta muy útil para el desarrollo de aplicaciones móviles modernas, ya que promueve la reutilización de componentes y un flujo de trabajo más eficiente.

Recomendaciones

- Practicar el uso de Modifiers para personalizar el espaciado, alineación y bordes de los elementos en Compose.
- Aprovechar los `@Preview` para visualizar los cambios en tiempo real y agilizar el desarrollo.
- Seguir explorando las guías oficiales de Android Developers para mantener buenas prácticas al trabajar con Jetpack Compose.
- Aplicar lo aprendido con Row, Column y Box en otros componentes como Card o LazyColumn para crear interfaces más completas.

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

[1] I. Sommerville, *Ingeniería del Software*, 9.ª ed. Madrid: Pearson Educación, 2011.

[2] R. S. Pressman, *Ingeniería del Software: Un enfoque práctico*, 7.ª ed. México: McGraw-Hill, 2010.

[3] Escuela Profesional de Ingeniería de Sistemas, «Guía de Prácticas de Laboratorio». 2024.

[4] Android Developers, «Compose layout basics». 2024.

[5] Android Developers, «Modifiers in Compose». 2024.

[6] DevTalles, «POSICIONA elementos con JETPACK COMPOSE de manera FÁCIL». 2023.

[7] DevTalles, «Usando Cards de Material Design en Jetpack Compose». 2023.